

# Analysis: Diagonal Puzzle

## Test set 1 (Visible)

For a square grid of size  $N$ , there's total  $2 \times N - 1$  diagonals that are parallel to the main diagonal, and  $2 \times N - 1$  diagonals that are perpendicular to the main diagonal.

For each of the diagonals parallel to the main diagonal, we can fix either we would flip them or not, and then check if each of the perpendicular diagonals contain cells of the same color. If all cells are the same for each of the perpendicular diagonals, then this would lead to a solution. If all white, just flip the whole diagonal, if not, nothing is necessary.

There are total  $2^{2 \times N - 1}$  possible combinations for flipping diagonals parallel to the main diagonal, and we can check if the perpendicular diagonals have all same coloured cells in  $O(N^2)$  time complexity, which leads to a total time complexity of  $O(2^{2 \times N - 1} \times N^2)$  per test case, which is sufficient for test set 1.

## Test set 2 (Hidden)

One interesting observation is that if we decide whether we would flip the two largest diagonals or not (in the case where  $N$  is odd, we decide whether we flip the largest diagonal in one direction, and the second largest diagonal in the other direction), we can pick all other diagonals to flip deterministically. The diagonals are shown for grids of size  $N = 6$  and  $N = 7$  below.

```
\...../ \...../
.\.../. \...../
..\.../. \...../
...\/... \...../
...\/... \...../
./...\. \...../
/.....\ \...../
      ./.....\
```

There are four choices for flipping/not flipping the two chosen diagonals. And for each of these choices, we can go through all other diagonals in the grid. For each of these other diagonals, we can check if the square where it intersected with one of the chosen diagonals is white. If it is white, we need to flip this diagonal, otherwise not. If after all the flips, the final grid is all black squares, then we have a possible answer. We take the minimum flips of the four possible choices.

There are 4 choices for the flipping of the two chosen diagonals. And for each of the choices, the flipping of all other diagonals, and finally checking for all black cells can be done in  $O(N^2)$  time, which leads to a total time complexity of  $O(N^2)$ , which is sufficient for test set 2.

An alternative solution is to convert this problem into a variant of [2-coloring](#). Each of the cells in the grid is shared by two diagonals. If a cell is white, either one of them needs to be flipped to make this cell black. Otherwise, none of these two diagonals needs to be flipped. For this problem, we can consider each diagonal as a vertex of a graph, and the squares in the grid are the edges between the two diagonals that affect that square. If the square is initially black, then the endpoints of the corresponding edge must be colored the same, else they must be colored differently. By color, we mean chosen to be flipped or not flipped. So we should color in

such a way that all conditions for all edges in the graph are satisfied and we have the minimum number of flips possible. This can be done with a DFS for each component of the graph.

There are total  $O(N^2)$  edges in the graph. Building the graph and solving it can be done in  $O(|Edges|)$ . This leads to a time complexity of  $O(N)$ , which is sufficient for test set 2.

# Analysis: Eleanagram

## Test set 1

We need to divide each digits to two partitions: positive partition and negative partition, where positive partition means the digit is on the odd index (be calculated as *add*), and negative partition means the digit is in the even index (be calculated as *minus*).

We can use dynamic programming to solve test set 1. Let  $dp[i][j][k]$  denote if it is possible to achieve the state that when we are considering digits 1, 2, ...  $i$ , the current number of digits in the positive partition is  $j$  and the current sum modulo 11 is  $k$ . Then for each digit  $i$ , we can put 0, 1, ...,  $A_i$  digits into the positive partition, and calculate if the current state is possible. We want to calculate  $dp[9][sum(A)/2][0]$ , where  $sum(A)$  means the total sum of all elements in array  $A$ .

The time complexity is  $O(9 * sum(A) * 11 * max(A))$ , which fits the time limit for test case 1. Here  $max(A)$  means the maximum of all elements in array  $A$ .

## Test set 2

Assume the positive number of digits  $i$  is  $P_i$ , and negative number of digits  $i$  is  $A_i - P_i$ . Then, we will have the following three equations:

$$\begin{aligned}(1) \quad \sum P_i &= \text{ceil}(\text{sum}(A) / 2) \\(2) \quad \sum i \times (P_i - (A_i - P_i)) \mod 11 &= 0 \\(3) \quad 0 \leq P_i \leq A_i\end{aligned}$$

In order to solve this, initially we can put half the number of each digits to be in positive partition (e.g.  $P_i = A_i / 2$ , take care of odd numbers), and then try to adjust each  $P_i$  to satisfy equation (2). For each  $i$ , we can adjust its  $P_i$  from  $-A_i / 2$  to  $A_i / 2$ .

We can prove the two following conclusions:

1. If there are at least two numbers of  $A_i \geq 10$ , then the solution must exist.

This is very easy to prove. We can only adjust these two digits from -5 to 5, and each adjustment will result in a different remain value of modulo 11. Thus, we will get 0 finally.

2. If there are at least three numbers of  $A_i \geq 6$ , then the solution must exist.

To prove this, we can prove that:

For any  $1 \leq i < j < k \leq 9$ ,  $0 \leq r \leq 10$ , the following equations:

$$\begin{aligned}(1) \quad (i * x_1 + j * x_2 + k * x_3) \mod 11 &= r \\(2) \quad x_1 + x_2 + x_3 &= 0 \\(3) \quad -3 \leq x_i \leq 3\end{aligned}$$

will have a valid solution.

This can be proved by iterating all possible situations, where the total number is  $9C3 * 11 * 7 * 7 = 45276$ , quite small.

According to these two conclusions, if there are at least two numbers  $\geq 10$  or at least three numbers  $\geq 6$ , we can return YES immediately. Otherwise, there are at most one value  $\geq 10$ , and at most two values  $\geq 6$ , we can calculate all possible situations, where in the worst case time complexity is  $O(6^7 * 10) = O(2799360)$  which fits the time limit for test case 2.

# Analysis: H-index

### Test set 1 (Visible)

For a set of papers, let's define a function  $\text{score}(x)$ , which denotes the number of papers in the set with citations at least  $x$ . We can calculate the score for every integer  $x \leq n$ , where  $n$  is the number of papers in the set. For each set of papers, the answer is maximum  $x$  such that  $\text{score}(x) \geq x$ . We can store the counts of each of the citation numbers in the input in an array, and calculate cumulative sum to find the score for any  $x$ .

We need to find the maximum  $x$  where  $\text{score}(x) \geq x$ , each time a new paper is added. We can find that using the approach mentioned above in  $O(\max(\mathbf{A}))$  time, where  $\max(\mathbf{A})$  is the maximum citation number in the set of papers. An observation here is, we can consider all citations  $\geq \mathbf{N}$  as  $\mathbf{N}$ , which allows us to find H-index for any set of papers in  $O(\mathbf{N})$ . This leads to a total time complexity of  $O(\mathbf{N}^2)$  per test case, which is sufficient for test set 1.

### Test set 2 (Hidden)

For test set 2, we initialize the H-index with 0, and after adding each paper, we need to update the current H-index. We can use a minimum priority queue data structure to store the citation numbers. As we go through each of the papers, we keep updating the current H-index. We also keep updating the priority queue so that it only contains numbers greater than the current H-index and remove the rest. For each new paper, we can follow these steps

1. If the current citation number is bigger than the current answer, add it in the priority queue
2. Remove all the citation numbers from the priority queue which is not greater than the current answer
3. If the size of priority queue is not less than the current answer + 1, increment the answer by 1

A single operation of priority queue takes  $O(\log \mathbf{N})$  time, and a number is added and removed at most once. This leads to a total time complexity of  $O(\mathbf{N} \times \log \mathbf{N})$  per test case, which is sufficient for test set 2.

A solution with linear time complexity is also possible, if we store the citation numbers in an array or a hashtable to calculate the answer. This is left as an exercise for the readers.